Trabajo Práctico: API REST de Personas y Pizzas

En este trabajo recibí la API que usa la pizzería para registrar a sus clientes y las pizzas que pide cada uno, y la tuve que revisar, corregir y ampliar. En este informe cuento qué encontré, qué cambié y por qué, y cómo lo comprobé. Acá incluyo las capturas más importantes; el resto está en la carpeta informe/capturas del repositorio.

1. Puesta en marcha y diagnóstico

Instalé el proyecto en Windows con Python 3.14 siguiendo el manual: creé el entorno virtual, instalé las dependencias, generé la base con build_database.py y levanté el servidor con app.py. Funcionó a la primera. La carpeta templates con home.html ya venía incluida en el material, así que tanto la página de inicio como Swagger abrieron sin problemas.

Después probé los nueve endpoints desde Swagger, cada uno con un caso que debía salir bien y con los casos de error que se me ocurrieron, y fui comparando el código que respondía la API con el que figura en el contrato (el archivo swagger.yml). Este es el resumen:

Endpoint	Dice el contrato	Devuelve la API	Coincide?
GET /people	200	200	Sí
POST /people	201	201, y también 406 y 400 que no figuran	Solo el caso normal
GET /people/{lname}	200	200, y 404 que no figura	Solo el caso normal
PUT /people/{lname}	200	201	No
DELETE /people/{lname}	204	200 con un texto en inglés	No
POST /pizzas	201	201, 404 que no figura y 500 si falta content	No
GET /pizzas/{pizza_id}	200	200, y 404 que no figura	Solo el caso normal
PUT /pizzas/{pizza_id}	200	201	No
DELETE /pizzas/{pizza_id}	204	204	Sí

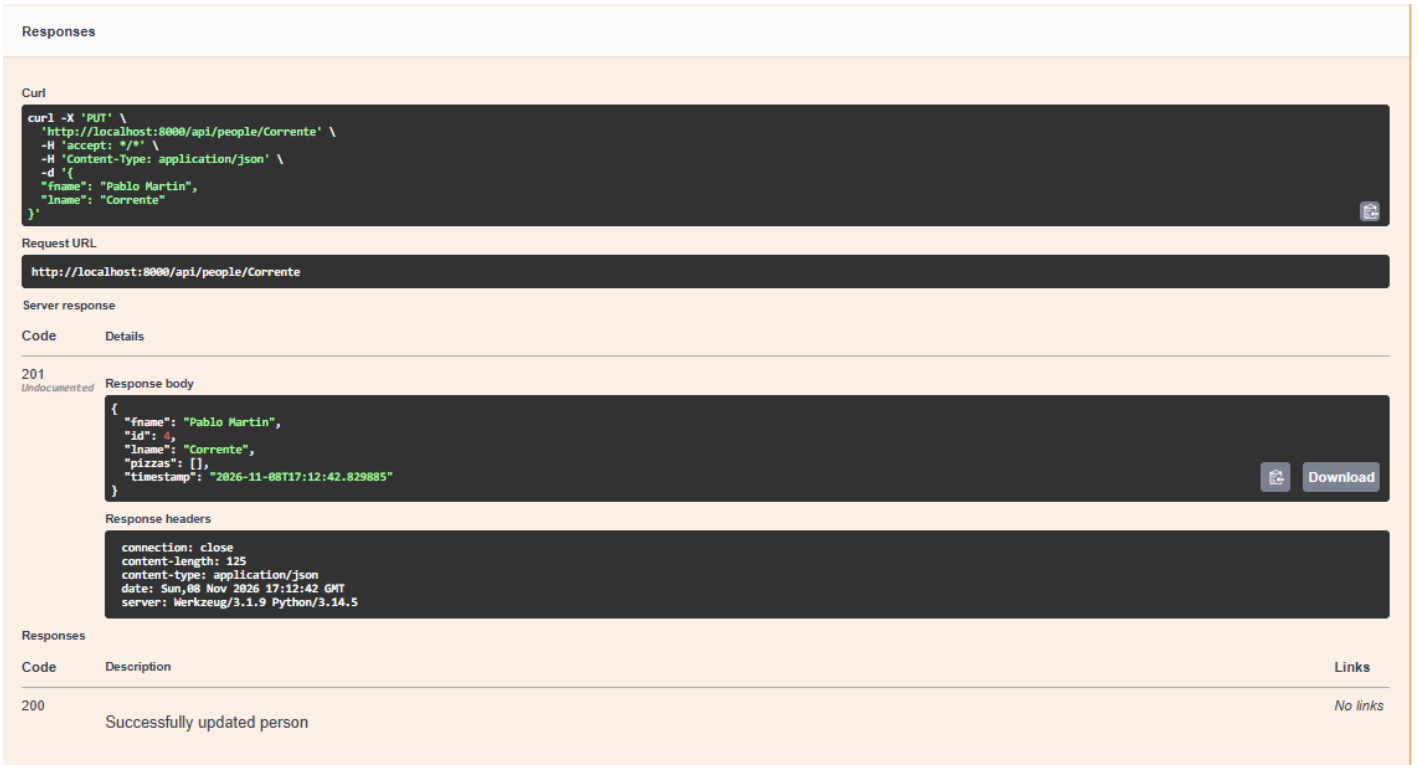
Las diferencias más claras fueron las de las modificaciones (PUT), que devuelven 201 cuando el contrato dice 200, y la del borrado de personas, que responde 200 con un texto en inglés en lugar de 204. Swagger mismo lo marca: debajo del código aparece la palabra “Undocumented” (capturas 1 y 2).

También noté que ningún error estaba documentado. La API responde 404 cuando no encuentra a alguien, 406 si el apellido ya existe y 400 si falta un dato obligatorio, pero en el contrato solo figuraba la respuesta exitosa de cada endpoint.

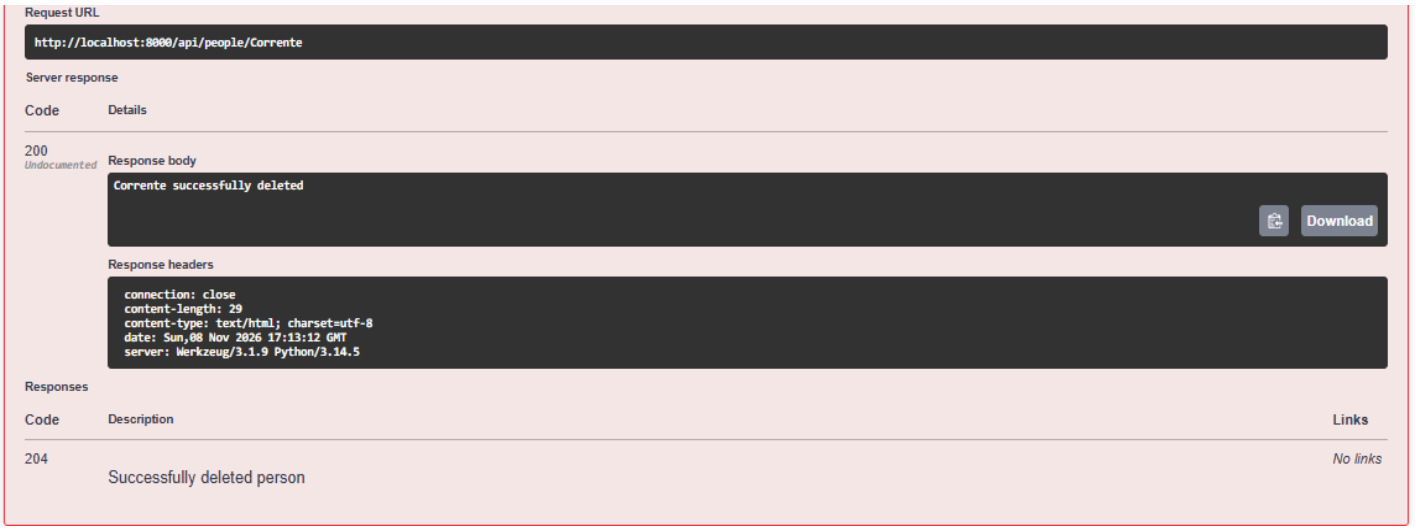
Lo más grave apareció al crear una pizza sin el campo content: el servidor se rompía con un error 500 y mostraba la página de depuración de Flask, donde se ven rutas de mi computadora y parte del código (captura 3). Pasaba porque el contrato no marcaba content como obligatorio y la base de datos sí. Algo parecido pasaba con

person_id: si no se mandaba, la respuesta era 404 “persona no encontrada”, cuando en realidad lo que faltaba era un dato.

Otras cosas que anoté: el PUT de personas obligaba a mandar el apellido en el cuerpo pero después no lo usaba; se podían crear personas sin apellido y pizzas sin contenido; no había forma de listar todas las pizzas, y los mensajes de error estaban en inglés mientras que el resto estaba en español.



Captura 1. PUT de una persona: responde 201, el contrato dice 200.



Captura 2. DELETE de una persona: 200 con texto, el contrato dice 204.

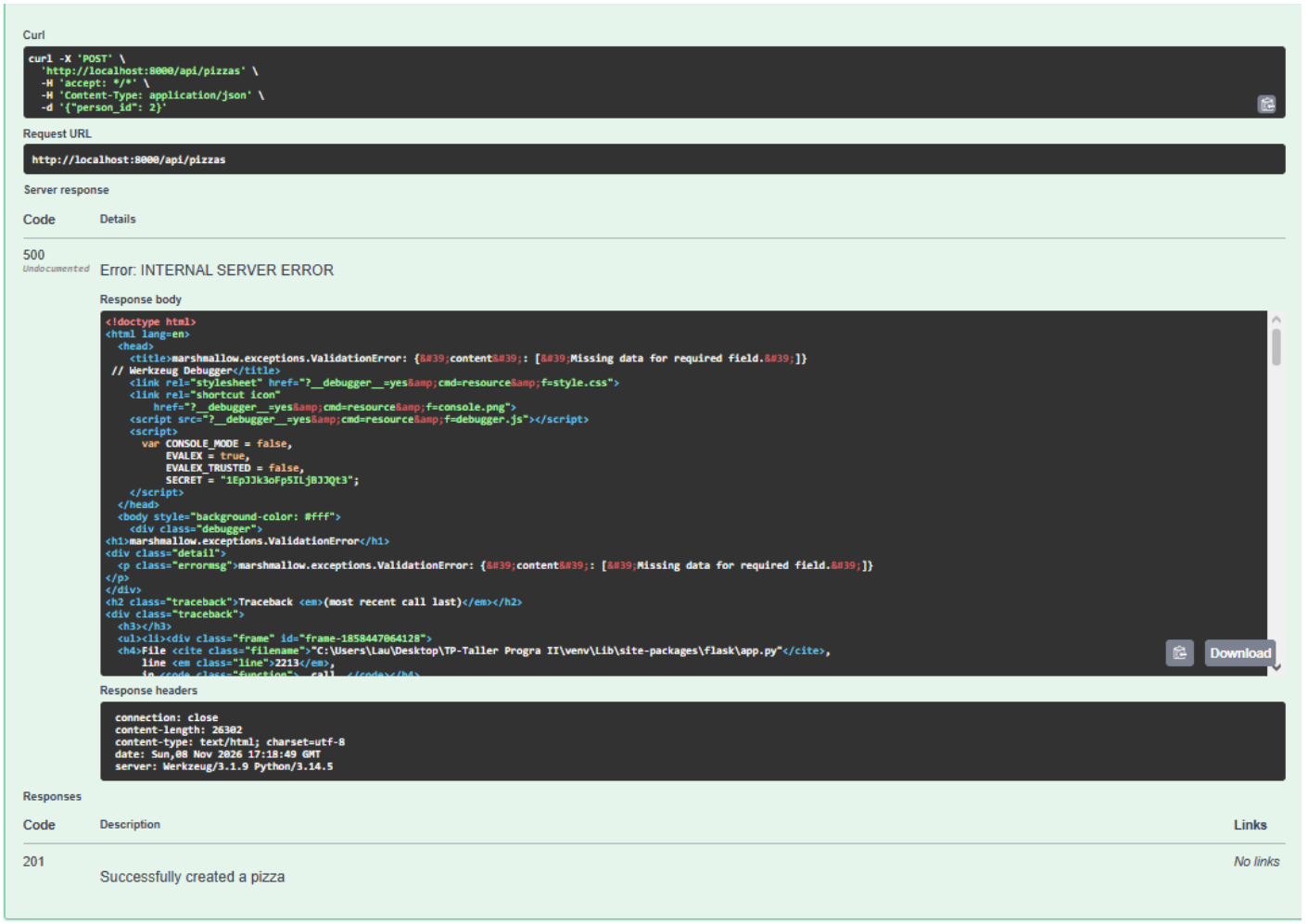
2. Alinear el contrato con el código

Para cada diferencia tuve que decidir si cambiaba el código o el contrato. El criterio fue quedarme con el que hacía lo correcto y corregir el otro.

En los PUT cambié el código para que devuelvan 200: el 201 significa que se creó algo nuevo, y una modificación no crea nada. En el borrado de personas también cambié el código, para que devuelva 204 sin texto, que es lo que pedía el contrato y lo que ya hacía el borrado de pizzas. Así los dos borrados se comportan igual.

Los errores, en cambio, los agregué al contrato, porque el código ya respondía bien y lo que faltaba era documentarlos. Escribí la descripción de cada error una sola vez, en la sección de componentes, y cada endpoint la referencia. De esa forma no tuve que repetir el mismo texto en los nueve endpoints.

Para el error 500 también toqué el contrato: marqué content y person_id como obligatorios. Ahora, si falta alguno, Connexion rechaza el pedido con un 400 antes de que llegue a mi código, y el servidor ya no se rompe (captura 4). En el PUT de personas dejé de pedir el apellido en el cuerpo y pido solo el nombre, que es lo único que se puede cambiar por esa ruta.



Captura 3. Antes: crear una pizza sin content daba error 500.



Captura 4. Después: el mismo pedido devuelve 400 con el motivo.

Definí un esquema Pizza con todos sus campos y lo usé en todos los endpoints de pizzas. Como para crear una pizza hacen falta la persona y el contenido, pero para modificarla alcanza con el contenido, uní el esquema con la lista de campos obligatorios de cada caso (con allOf), sin tener que copiarlo dos veces. Los campos que completa el servidor solo, que son el id y la fecha, quedaron como de solo lectura.

Por último hice los dos cambios que pide la consigna en el código: reemplacé Query.get() por db.session.get(), y datetime.utcnow por datetime.now(timezone.utc), que es la forma actual de obtener la hora (la anterior está desaconsejada desde Python 3.12). Con todo esto, al volver a probar, ningún código aparece como "Undocumented" (captura 5).

3. Endpoints nuevos

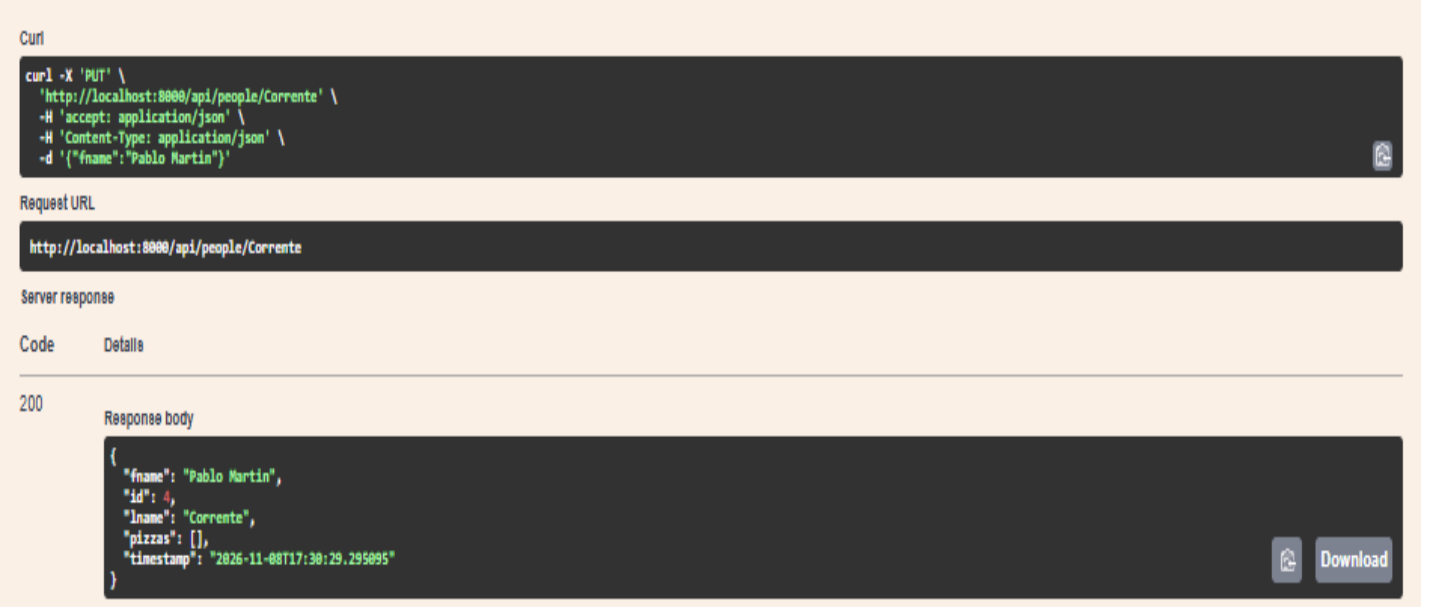
Para cada endpoint nuevo primero lo escribí en swagger.yml y después programé la función. Los tres son de consulta.

GET /api/pizzas lista todas las pizzas. Si se le pasa person_id, muestra solo las de esa persona, y si ese número no corresponde a nadie devuelve 404.

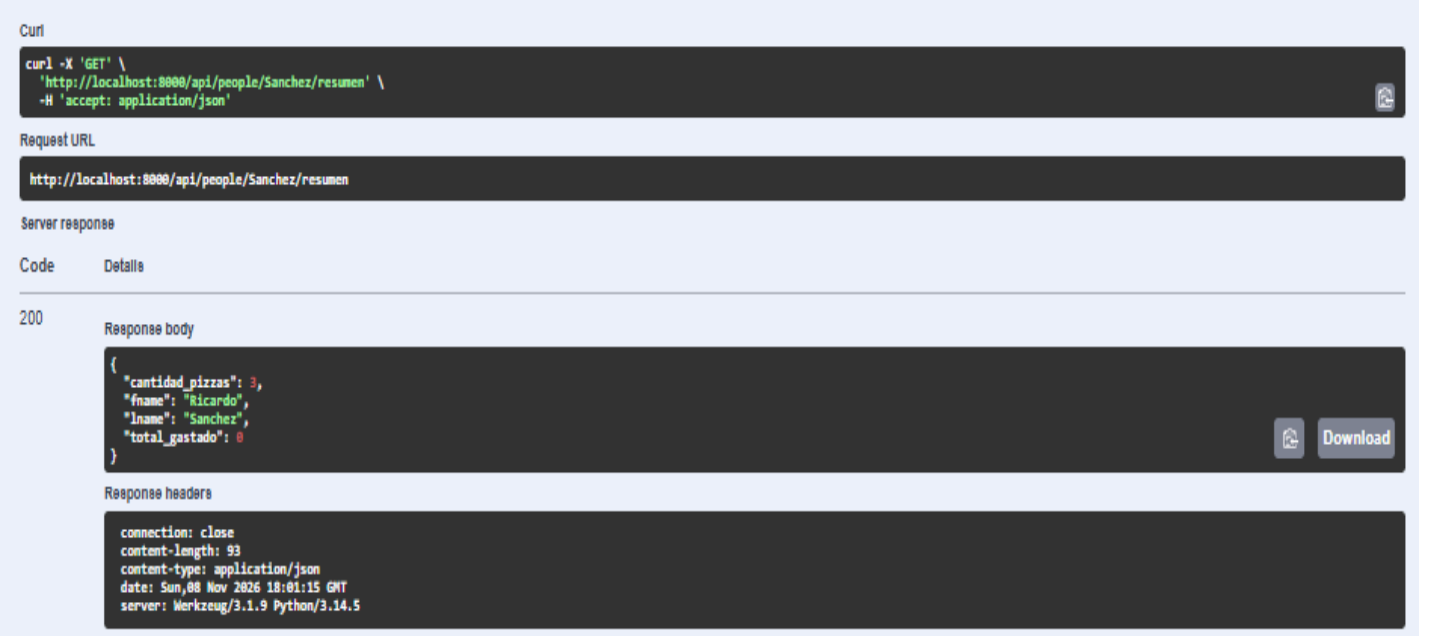
GET /api/people/{lname}/pizzas devuelve las pizzas de una persona, o 404 si el apellido no existe.

GET /api/people/{lname}/resumen devuelve el nombre, el apellido, cuántas pizzas pidió esa persona y cuánto gastó en total (captura 6), o 404 si no existe.

Mientras los hacía me di cuenta de que casi todas las funciones repetían lo mismo: buscar a la persona y, si no estaba, cortar con un 404. Lo pasé a una función buscar_persona, e hice lo mismo con las pizzas. Así cada endpoint quedó en dos o tres líneas y el mensaje de error está escrito en un solo lugar.



Captura 5. PUT de una persona después de la corrección: 200.



Captura 6. Resumen de Sanchez: 3 pizzas y el total gastado.

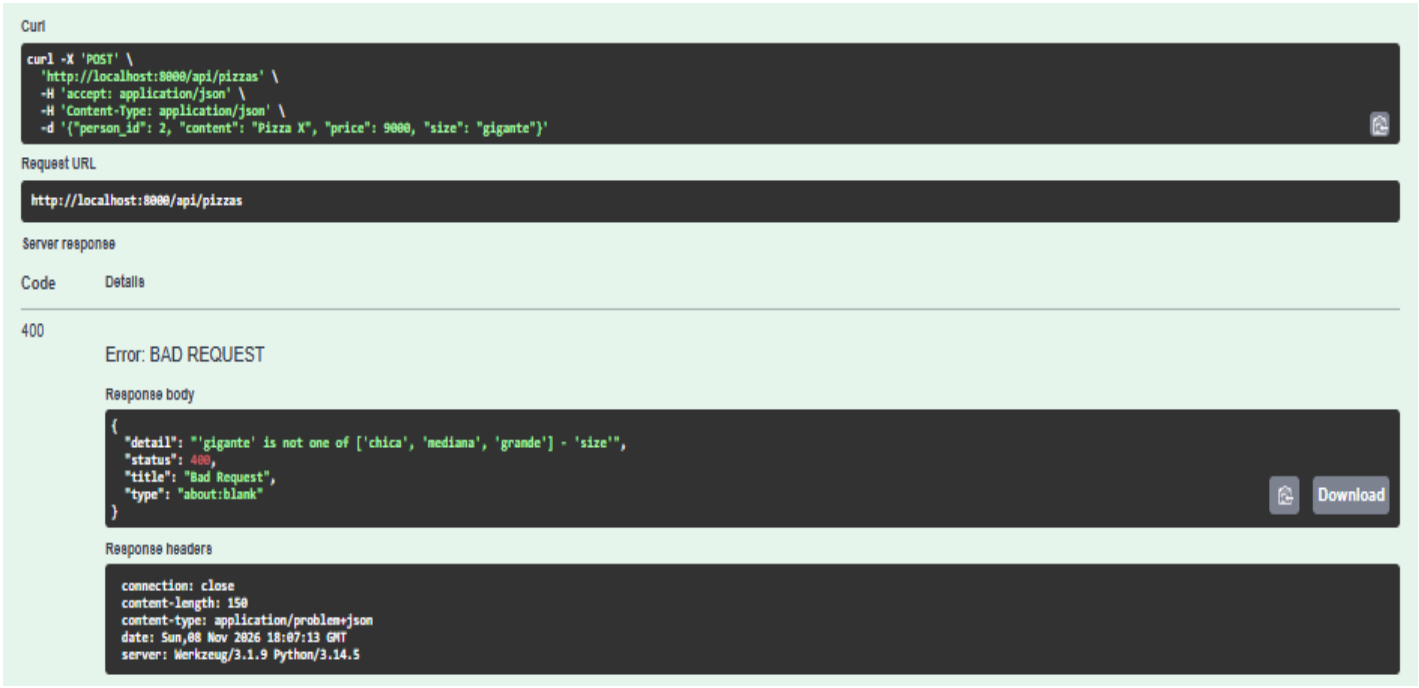
4. Precio y tamaño de las pizzas

A las pizzas les agregué dos datos: el precio, que es obligatorio y tiene que ser mayor que cero, y el tamaño, que solo puede ser chica, mediana o grande. Los agregué como columnas nuevas en models.py, y las reglas las puse en el contrato: un mínimo para el precio y una lista cerrada de valores para el tamaño. Gracias al esquema Pizza de la parte 2, el cambio lo hice en un solo lugar y se aplicó a todos los endpoints de pizzas.

No tuve que escribir ningún if para validar estos datos. Connexion compara lo que llega con el contrato, y si el precio es negativo, es cero o no es un número, o si el tamaño no es uno de los tres permitidos, responde 400 (capturas 7 y 8). Como la consigna dice que el precio es obligatorio pero no dice lo mismo del tamaño, dejé el tamaño como opcional.



Captura 7. Precio negativo: 400.



Captura 8. Tamaño "gigante": 400.

En build_database.py les puse precio y tamaño a todas las pizzas de ejemplo y corregí los errores de tipeo: "Piza Cuatro Quesos", "roqefort", "Peperoni" y las tildes de Jamón y Rúcula. Como el script borra la base y la vuelve a crear, hubo que correrlo de nuevo para que la tabla tuviera las columnas nuevas. Con los precios cargados, el resumen ya suma el total real: para Sanchez da 28.500, que es 9.500 + 8.200 + 10.800.

5. Mensajes de error y pruebas automáticas

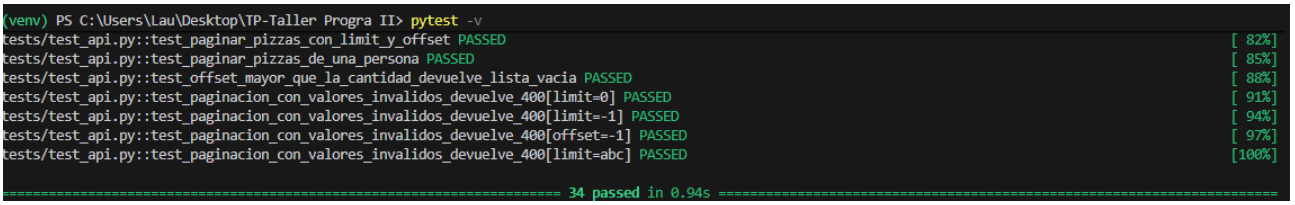
Pasé al español todos los mensajes de error de people.py y pizzas.py, por ejemplo "No existe una persona con apellido Gomez". Los mensajes de los errores 400 siguen en inglés porque no los escribo yo: los genera

Connexion cuando compara el pedido con el contrato, y para traducirlos habría que reemplazar esa parte de la librería.

Para no aceptar textos vacíos agregué en el contrato que el apellido y el contenido de la pizza tengan al menos un carácter y que no sean solo espacios. Con eso, crear una persona con apellido vacío o una pizza sin descripción devuelve 400.

Las pruebas automáticas están en tests/test_api.py y se corren con el comando pytest. Usan el cliente de prueba de Flask, que hace los pedidos a la API sin necesidad de levantar el servidor. Antes de cada prueba se arma una base nueva en memoria con los datos de ejemplo, así las pruebas no tocan people.db y una no afecta a la otra. Para eso hice que config.py pueda recibir otra base de datos desde afuera, y que build_database.py tenga una función que reutilizan las pruebas.

Las pruebas cubren todo lo que pide la consigna: listar y crear personas, apellido duplicado, persona inexistente, crear una pizza, pizza para una persona que no existe, precio inválido, borrado en cascada y los tres endpoints nuevos. Además agregué casos de tamaño inválido, textos vacíos, modificaciones y borrados. En total son 34 y pasan todas (captura 9). Agregué pytest a requirements.txt y escribí el README con los pasos para instalar, levantar la API y correr las pruebas.

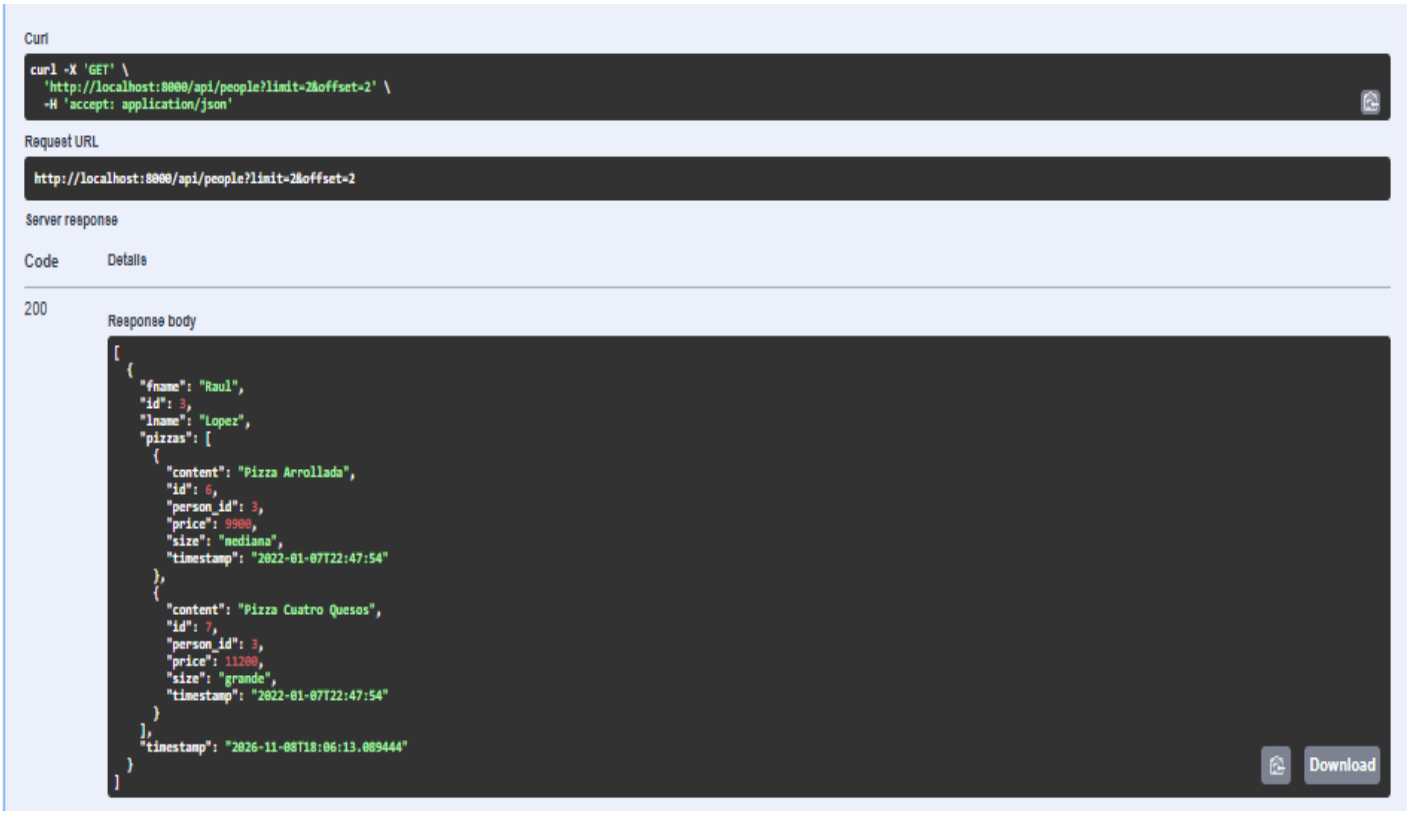


Captura 9. Resultado de pytest: 34 pruebas correctas.

6. Parte opcional: paginación

Elegí la paginación. Ahora GET /api/people y GET /api/pizzas aceptan dos valores opcionales: limit, que es cuántos resultados traer, y offset, que es cuántos saltear desde el principio. Por ejemplo, /api/people?limit=2&offset=2 saltea a Sanchez y Gomez y devuelve solo a Lopez (captura 10). Si no se mandan, los endpoints devuelven todo, igual que antes. Ordené los resultados por id para que una página no se mezcle con la siguiente.

Algo que me llamó la atención: si en Swagger ponía limit = 0, la página no me dejaba enviar el pedido, porque Swagger también lee las reglas del contrato. Para comprobar que el servidor lo rechaza por su cuenta, escribí la dirección directamente en el navegador y respondió 400.



Captura 10. GET /api/people con limit=2 y offset=2: solo Lopez.

7. Dificultades

Al principio me costó distinguir en Swagger lo que dice la documentación de lo que responde realmente el servidor. La sección Responses que aparece al abrir un endpoint es lo que promete el contrato, y la respuesta real recién aparece al apretar Execute. Una vez que lo entendí, el diagnóstico fue mucho más fácil.

En una de las pruebas del error 400 me salía un mensaje distinto del que esperaba: había dejado una coma al final del JSON, y la API lo rechazaba por estar mal escrito y no porque faltara el apellido. En otra cambié el valor del campo pero no apreté Execute, y la captura mostraba la respuesta anterior.

8. Herramientas utilizadas

Para este trabajo usé Claude como apoyo: me fue explicando cada parte, me ayudó a escribir y revisar el código y a ordenar este informe. Todos los cambios los probé yo en mi computadora, con Swagger y con las pruebas automáticas.

LINK DEL GITHUB: https://github.com/LautaroSalvador/TP_Salvador_Lautaro